Migrate an app

Migrate an existing private app to the projects framework

Learn how to migrate an existing private app to the projects framework.

If you previously built a **private app on the previous version (2025.1)** of the developer framework, you can manually migrate your app's configuration to the latest version (2025.2).

Key limitations

Keep the following limitations in mind before migrating your private app:

Limited support for serverless functions:

> If your existing private app includes serverless functions with core logic (e.g., the function transforms data that isn't contained in the backend), you should not migrate it to the new developer platform until a full migration path is released.
>
> > If needed, you can always **build a net-new app** on version 2025.2 of the developer platform alongside your existing app.
> >
> > Creating a **project-built private app** using version 2025.1 of the developer platform with **serverless functions** is still supported.
>
> Otherwise, if your serverless functions are only acting as a proxy to your backend, you can proceed with migration by following the guidance in the **section below** on using `hubspot.fetch()` after completing the previous steps in this guide.

> For more information on serverless function support and how to leverage third-party hosting for your backend service, check out **this post** on the HubSpot Developer Blog.

Other unsupported features:

sure you disable the integration before you begin the migration. To disable the GitHub integration and setup GitHub Actions for automating CI/CD, check out the instructions in **this article**.

Summary of the migration process

This guide will walk you through the following:

You'll start by cloning your existing project so the original files can serve as a backup, then you can review each of the associated configuration files to ensure they conform to the new project schema. This will require minor updates to the names and structure of your component configuration files, as well as their respective properties.

You'll then update your existing top-level `hsproject.json` and `app.json` configuration files.

You can then follow each of the subsequent sections to update the configuration for your app components to conform to version `2025.2` of the developer platform, based on the existing features you had set up (e.g., an app card built using UI extensions).

Note that, with the exception of the `hsproject.json` file, all other configuration files follow a predictable naming scheme (`*-hsmeta.json` where the `*` is based on the specific directory or component) and all share the same top-level properties:f

```
{
  "uid": "example-unique-identifier",
  "type": "example-component",
  "config": {
    ...
  }
}
```

HubSpot Developers

›

Clone your existing project configuration files

Before making any updates, clone your existing project so you have a backup you can reference or fall back to in case you run into issues. After you've cloned your project, open the cloned project files in your preferred IDE, such as VSCode.

If you're looking for a minimal project that conforms to the new `2025.2` schemas that you can reference, you can download a boilerplate template:

Make sure you've installed the latest version of the HubSpot CLI by running the command `npm install -g @hubspot/cli@latest` and connected it to your account using `hs account auth` command. If you haven't installed the HubSpot CLI yet, run `npm install -g @hubspot/cli`. You should be on `v7.6.0` of the CLI before proceeding.

Run the command below in your terminal to create a project with the boilerplate template for an app with `private` distribution and `static` auth.

```
hs project create --distribution private --auth static --pr
```

Follow the prompts to provide a name and local directory to download the boilerplate project into, along with any .

Open the newly created project in your preferred editor. You can then compare the project directory structure and the associated `*-hsmeta.json` schema files to your existing project to ensure the specifications match when applicable.

A full reference of the new project structure for version `2025.2` of the developer platform is detailed in the guide.

Changes to hsproject.json config

The changes to the top-level `hsproject.json` involve minor changes to the `name` and `platformVersion` properties, as outlined in the code blocks below:

```
  "srcDir": "src",
  "platformVersion": "2025.1"
}
```

After:

```
{
  "name": "My new migrated app (Developer platform v2025.2)",
  "srcDir": "src",
  "platformVersion": "2025.2"
}
```

The top-level `hsproject.json` file in your project lives in the same location in the new developer platform, but you'll need to update the `platformVersion` to `"2025.2"`. You may also want to update the `name` field with a unique name so that it doesn't override your existing project when you upload it. For example, if the name of your existing private app was named `My private app`, you might want to append `(Developer Platform v2025.2)` or something similar to distinguish it from the old app.

Review and update your app's top-level schema

The code blocks below provide examples of the config before and after the required changes:

Before (`src/app/app.json`):

HubSpot Developers

```
  "uid": "my-old-private-app",
  "public": false,
  "extensions": {
    "crm": {
      "cards": [
        {
          "file": "extensions/example-card.json"
        }
      ]
    }
  },
  "webhooks": {
    "files": "./webhooks/webhooks.json"
  }
}
```

After (`src/app/app-hsmeta.json`):

HubSpot Developers

```
    "description": "Example of migrated app on the new developer platform.",
    "name": "My new migrated app (Developer platform v2025.2)",
    "distribution": "private",
    "auth": {
      "type": "static",
      "requiredScopes": ["crm.objects.contacts.read", "crm.objects.contacts.wri
      "optionalScopes": [],
      "conditionallyRequiredScopes": []
    },
    "permittedUrls": {
      "fetch": ["https://api.example.com"],
      "iframe": [],
      "img": []
    },
    "support": {
      "supportEmail": "support@example.com",
      "documentationUrl": "https://example.com/docs",
      "supportUrl": "https://example.com/support",
      "supportPhone": "+18005555555"
    }
  }
}
```

On the older version of the developer platform, the configuration for your private app was specified in your `app.json` file. These config details for your app are now specified with your app schema file in the `/src/app/app-hsmeta.json` file. The key changes between your old `app.json` config and the new `app-hsmeta.json` config include the following:

- The top-level `public` property has been replaced with `distribution` and should be set to `private`. Note that the `type` sub-property of the `auth` field should be set to `static`, which will restrict installation of your app to a single account. Learn more about app distribution and authentication in the guide.
- Your app's scopes are now specified as a sub-property of the `auth` field, and are split out between `requiredScopes`, `conditionallyRequiredScopes`, and

previously configured UI extensions (e.g., cards on the CRM record page) are managed using the `cards/` directory of your project. Within that directory, card configuration details are specified in a `*-hsmeta.json` file, alongside the component code for your card provided in a `.jsx` file that's referenced using the `entrypoint` property of the `*-hsmeta.json` file.

You also don't need to define over the top-level `webhooks` property from your previous project in the new `app-hsmeta.json` file, as webhooks are configured and managed using the `webhooks/` directory of your project. Learn more in the **migrate webhook subscriptions section** below.

Update individual component configuration

These sections below outline how to port any UI extensions and webhooks over to your new app. If your old app had neither of these components, you can jump to the **upload your project** section.

Migrate CRM cards built with UI extensions

The code blocks below provide examples of the config before and after the required changes:

Before (`src/app/extensions/card.json`):

HubSpot Developers

```
    "uid": "example_app_card_private_static",
    "location": "crm.record.tab",
    "module": {
      "file": "example-app-card.jsx"
    },
    "objectTypes": [
      {
        "name": "contacts"
      }
    ]
  }
}
```

After (`src/app/cards/NewCard-hsmeta.json`):

```
{
  "uid": "example_app_card_private_static",
  "type": "card",
  "config": {
    "name": "example app card",
    "description": "Provides detailed information about a contact or company.",
    "previewImage": {
      "file": "./full-preview.png",
      "altText": "This describes the image"
    },
    "entrypoint": "/app/cards/example-app-card.jsx",
    "location": "crm.record.tab",
    "objectTypes": ["CONTACT"]
  }
}
```

CRM cards are now configured within the `cards/` directory of your project, replacing the old `extensions/` directory from your old project. Within the new `cards/` directory, card configuration details are specified in a `*-hsmeta.json` file, alongside the component

your legacy `app.json` info the associated properties in the `*.hsmeta.json` file in the `cards/` directory, keeping the following changes in mind:

- The value of the `type` property has been changed from `"crm-card"` to `"card"`.
- The `uid` property has been moved up from a sub-property of the `data` field and is now specified at the top-level of your config.
- The `data` property has been changed to `config`, which includes the following sub-properties:
  - The `title` property has been renamed to `name`.
  - A new `description` property allows you to provide more context around the functionality of your card. The description will appear in your app's project settings.
  - The `module` property has been renamed to `entrypoint` and the value should now be a string that represents the path to your JSX component, relative to the root of your project (e.g., `"/app/cards/example-app-card.jsx"`).
  - The `objectTypes` property has been simplified and is now an array of strings representing the object types where your card should appear (e.g., `["CONTACT", "COMPANY"]`).
  - The `location` property remains unchanged, and can be set to `crm.record.tab`, `crm.record.sidebar`, `crm.preview`, or `helpdesk.sidebar`.

If you **downloaded** the boilerplate project template, an example `example-app-card-hsmeta.json` config file and `example-app-card.jsx` JSX component are provided in the `src/app/cards` directory.

For a full guide on creating app cards on the new developer platform, check out **this article**.

Migrate webhook subscriptions

The code blocks below provide examples of the config before and after the required changes:

HubSpot Developers

```
      "targetUrl": "https://example.com/webhook",
      "maxConcurrentRequests": 10
    },
"subscriptions": {
      "crmObjects": [
        {
          "subscriptionType": "object.creation",
          "objectType": "contact",
          "active": true
        }
      ],
      "legacyCrmObjects": [
        {
          "subscriptionType": "contact.propertyChange",
          "propertyName": "lastname",
          "active": true
        }
      ],
      "hubEvents": [
        {
          "subscriptionType": "contact.privacyDeletion",
          "active": true
        }
      ]
}
```

After (`src/app/webhooks/webhooks-hsmeta.json`):

Please note:

The default `targetUrl` in the `webhooks.json` boilerplate configuration is set to `http://example.com/webhook` , so you'll need to update the `targetUrl` to point to a URL for a backend service that you own.

Each of the example subscriptions in `webhooks.json` is set to `active` by default. If you're not ready to set up webhooks when you first upload your project, set each of the `active` properties to `false` , or remove the `webhooks` directory entirely before you upload your project to HubSpot.

HubSpot Developers

```
    "settings": {
      "targetUrl": "https://example.com/webhook",
      "maxConcurrentRequests": 10
    },
    "subscriptions": {
      "crmObjects": [
        {
          "subscriptionType": "object.creation",
          "objectType": "contact",
          "active": true
        }
      ],
      "legacyCrmObjects": [
        {
          "subscriptionType": "contact.propertyChange",
          "propertyName": "lastname",
          "active": true
        }
      ],
      "hubEvents": [
        {
          "subscriptionType": "contact.privacyDeletion",
          "active": true
        }
      ]
    }
  }
}
```

Webhook subscriptions are still managed within a `webhooks/` directory of your project. Within the directory, subscription details are specified in a `*-hsmeta.json` file. The structure of the file is largely similar to the previous `webhooks.json` schema in your private app, with the following notable changes:

- A required `uid` property must be defined at the top-level of your `*-hsmeta.json` file, which should be given a name to differentiate it from other app features (e.g., `"migrated_private_app_webhooks"`).

top-level of your `*-hsmeta.json` file.

For a full reference on defining webhook subscriptions on the new developer platform, check out **this article**.

Upload your project

After you've migrated over the configuration from your existing private app into the respective sub-directories of your project, you can upload your new app to your HubSpot account. From there, you can find your app's access token that you can use to authenticate API requests and continue to build out functionality on the new developer platform.

> **Please note:**
>
> If your existing private app had any serverless functions defined:
>
> 1. Make a backup of the `src/app/app.functions` from your old project, along with any associated references to your serverless functions elsewhere in your project.
> 2. Serverless functions should not be included in the project directory that you upload as part of your new app. Once you've uploaded your project, you can follow the steps in the **section below** to recreate the same behavior on the new developer platform.

To upload your new project, run the following CLI command:

```
hs project upload
```

Migrate serverless function handling

> **Please note:**
>
> This section provides guidance on how to use the `hubspot.fetch()` function to migrate existing serverless functions that were acting purely as a proxy to your backend service.
>
> If your existing private app includes serverless functions with core logic (e.g., the function transforms data that isn't contained in the backend), you should not migrate it to the new

HubSpot Developers

based backend service and use the `hubspot.fetch()` **API** to fetch data. This will require you to migrate any existing service logic that was previously defined in your HubSpot-hosted serverless functions, as well as your private app access token to a third-party hosting platform, such as Vercel, DigitalOcean, AWS, etc.

To migrate your serverless function logic to a third-party hosting platform:

Locate your serverless functions in the project of your existing private app in the `src/app/app.functions` directory.

Copy all the relevant logic from your functions. In the example serverless function below, only line 4 would need to be copied over.

```
exports.main = async (context = {}) => {
  const { text } = context.parameters;

  const response = `This is coming from a serverless function!  You
entered: ${text}`;

  return response;
};
```

In the third-party hosting platform, paste in the logic from your previous serverless function definition, and ensure that any parameter names align. You'll need to consult the documentation for defining serverless function on the platform you're using.

Copy your access token from your app's **project details page** and add it as an environment variable with your third-party hosting platform so it can be referenced in your code.

Next, you'll need to update the `permittedUrls` property of your top-level `app-hsmeta.json` **schema file** to include the `fetch` field. The value of this field should be set to an array that includes the URL of your endpoint hosted in your third-party hosting platform.

Then, update any references in your app card React code to call the new serverless function URL you set up. You can learn more about using `hubspot.fetch()` in **this**

```
const { response } = await runServerless({
  name: "myFunc",
  parameters: { text: text },
});
```

Then you'd want to update the code in your new project to something like:

```
const response = await hubspot.fetch("https://my-new-server
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ text: "hello" }),
});
```

Clean up

After you've successfully uploaded your new project, migrated over any serverless function handling (if applicable), and fully tested your app to confirm the behavior is consistent, you can delete your old project within your HubSpot developer account. Remember that you still have the original backup of your project locally, as outlined in the of this guide in case you ever need it as a reference.

To delete your old project from your developer account:

In your HubSpot account, navigate to **Development**.

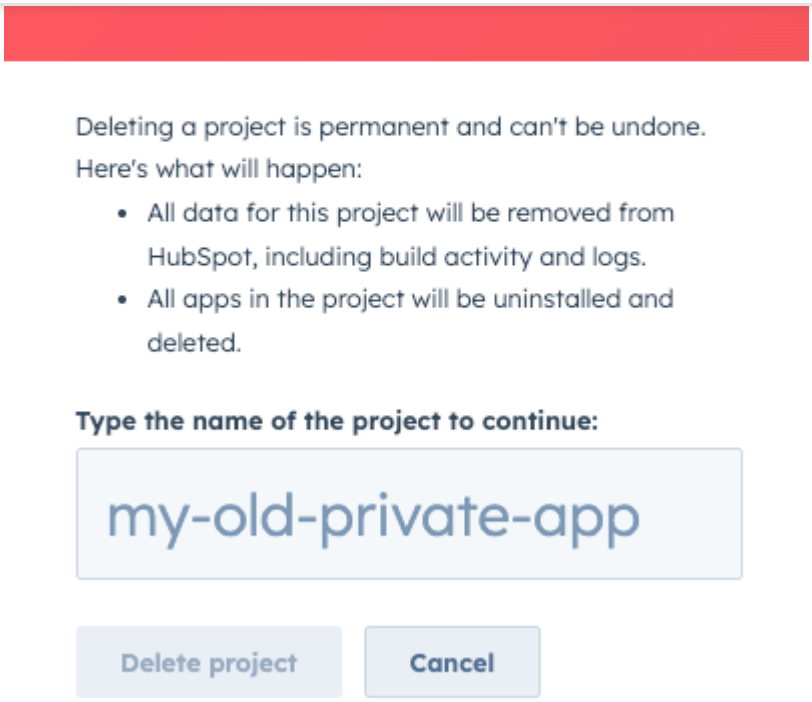
On the **Projects** page, click the **name** of your old project.

Click the **Settings** tab.

Under *Delete this project*, click **Delete [project name]**.

Review the information in the dialog box to confirm that you're ready to proceed. Then, enter the name of your project, and click **Delete project**.

HubSpot Developers



Next steps

Now that you've manually migrated over the components and configuration from your old private app, you can continue building features on the new developer platform by checking out these follow-up guides:

- Create a settings component
- Define a custom workflow action
- Manage your app in HubSpot

Last modified on January 28, 2026

Was this page helpful? Yes No